6.8610 Spring 2026 Lecture 3 Notes: Recurrent Neural Networks, Seq2Seq, and Attention

Junru Ren with help from Codex

February 10, 2026

1 Recurrent Neural Networks, Seq2Seq, and Attention

1.1 Motivation

The lecture introduces **recurrent neural networks (RNNs)** as the first neural architecture in the course that is explicitly designed for sequential data. The immediate motivation is that feed-forward models with fixed windows are poorly matched to language: they do not retain state across predictions, they scale poorly with larger contexts, and they struggle with long-range dependencies.

- Language is fundamentally sequential: characters form words, words form sentences, and sentences form larger discourse units.
- Many important linguistic phenomena depend on long-range context. For example, pronoun choice in "Emily . . . proud of her" versus "Miguel . . . proud of him" depends on a word many tokens earlier.
- Feed-forward models can use nonlinearities, but they still rely on fixed context windows and do not maintain memory from one prediction to the next.
- RNNs address this by carrying forward a hidden state that summarizes prior context.
- Once RNNs are available, they can be extended from next-token prediction to more general sequence modeling, including variable-length input/output tasks such as machine translation.
- This leads naturally to **sequence-to-sequence (seq2seq)** models and then to **attention**, which solves the bottleneck of compressing the entire source sentence into a single vector.

1.2 Key Concepts

Definition: Word Token vs. Word Type

A **word token** is a specific occurrence of a word in text, while a **word type** is the unique lexical form. For example, in "I ran and ran and ran" there are 6 word tokens but only 3 word types.

- **Type-based vs. token-based representations:**
  - Word2vec-style embeddings are type-based: every occurrence of a word like "bank" shares one representation.
  - RNN hidden states are token-based or contextualized: each occurrence of a word gets a context-specific representation.
- **Feed-forward language models are limited:**
  - fixed input length,

  - no state carried across predictions,
  - parameter count grows with window size,
  - no natural parameter sharing across positions,
  - effectively bag-of-words unless position is encoded explicitly.
- **Masked vs. autoregressive feed-forward modeling:** with a fixed window, a feed-forward network can either predict a missing token from surrounding context or predict the next token from only the left context. The autoregressive version is the one that directly defines a left-to-right sequence model.
- **RNN core idea:** use the previous hidden state together with the current input to compute the new hidden state.
- **Hidden state as meaning summary:** the hidden representation is meant to encode the meaning of the sequence seen so far.
- **Teacher forcing:** during training, the model receives the true previous token rather than its own predicted token.
- **Backpropagation through time (BPTT):** training an RNN requires propagating gradients backward through the unrolled sequence.
- **Truncated BPTT:** for efficiency, backpropagation may be limited to a finite recent window while the forward pass still carries recurrent state.
- **Sequence-to-sequence (Seq2Seq):** use one RNN as an encoder for the source sequence and another as a decoder for the target sequence.
- **Attention:** let each decoder step compute a weighted summary of all encoder hidden states, rather than relying only on the final encoder state.

1.3 Core Models and Equations

Feed-forward language model recap. For a fixed context representation X,

$$h = \phi(WX + b_h)$$

$$\hat{y} = \mathrm{softmax}(Uh + b_y)$$

This allows nonlinear combination of a fixed window, but it does not maintain sequential state.

Feed-forward masked vs. autoregressive objectives. With a fixed-size window, one can train the same feed-forward architecture in two different ways:

- **Masked language modeling:** hide a token and predict it from both left and right context,

$$p_\theta(x_t \mid x_{t-k:t-1}, x_{t+1:t+k}).$$

- **Autoregressive language modeling:** predict the next token from only the previous context,

$$p_\theta(x_t \mid x_{t-k:t-1}).$$

The lecture explicitly contrasted these two views: a masked objective can learn useful contextual representations, but only the autoregressive formulation directly supports a normalized left-to-right probability for the whole sequence $p(x_{1:T})$.

Basic RNN update. For input embedding x_t, hidden state h_t, and output distribution $\hat{y}_t$,

$$h_t = f(Wx_t + Vh_{t-1} + b_h)$$

$$\hat{y}_t = \text{softmax}(Uh_t + b_y)$$

with initial state typically set to

$$h_0 = \mathbf{0}.$$

Cross-entropy (CE) loss at each timestep.

$$\text{CE}(y_t, \hat{y}_t) = -\sum_{j \in V} y_{t,j} \log \hat{y}_{t,j}$$

where V is the vocabulary and y_t is the one-hot gold distribution.

Sequence loss.

$$\mathcal{L} = \sum_{t=1}^{T} \text{CE}(y_t, \hat{y}_t)$$

Backpropagation through time intuition. Because h_t depends recursively on earlier states, gradients to earlier recurrent weights involve long chains of multiplications. Abstractly,

$$\frac{\partial \mathcal{L}_T}{\partial h_k} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{t=k+1}^{T} \frac{\partial h_t}{\partial h_{t-1}}$$

which explains why gradients may vanish or explode.

Gradient clipping. If gradient norm exceeds threshold τ, rescale:

$$g \leftarrow g \cdot \min\left(1, \frac{\tau}{\|g\|}\right)$$

This preserves direction while limiting magnitude.

Seq2Seq encoder-decoder formulation.

- The encoder consumes source tokens $x_1, \ldots, x_N$ and produces hidden states $h_1^{enc}, \ldots, h_N^{enc}$.
- In the basic seq2seq model, the final encoder hidden state initializes the decoder:

$$h_0^{dec} = h_N^{enc}$$

- The decoder then produces target tokens $y_1, \ldots, y_M$ autoregressively:

$$h_t^{dec} = f(W_y y_{t-1} + V h_{t-1}^{dec} + b)$$

$$\hat{y}_t = \text{softmax}(U h_t^{dec} + b_y)$$

Attention scores. For decoder state h_t^{dec} and encoder state h_i^{enc}, compute a raw compatibility score

$$e_{t,i} = \text{score}(h_t^{dec}, h_i^{enc})$$

where the score function can be a feed-forward network, a dot product, or another learned similarity function.

Popular attention scoring functions. Common choices include:

- **Additive / concat scoring:**

$$e_{t,i} = v_a^\top \tanh\Big(W_a[h_t^{dec}; h_i^{enc}]\Big).$$

- **Bilinear / general scoring:**

$$e_{t,i} = (h_t^{dec})^\top W_a h_i^{enc}.$$

- **Dot-product scoring:**

$$e_{t,i} = (h_t^{dec})^\top h_i^{enc}.$$

The transcript emphasized the tradeoff between flexibility and efficiency: more expressive learned scorers are possible, but dot-product scoring became especially attractive because it is simple, fast, and works well at scale.

Attention weights.

$$a_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{N} \exp(e_{t,j})}$$

These weights form a distribution over source positions for each decoder step t.

Context vector.

$$c_t = \sum_{i=1}^{N} a_{t,i} h_i^{enc}$$

This is a weighted summary of the encoder hidden states.

Decoder with attention. The output at decoder step t is based on both the current decoder state and the context vector:

$$\hat{y}_t = g(h_t^{dec}, c_t)$$

where g may be implemented by concatenation followed by an affine transformation and softmax.

Historical machine-translation framing. Before neural seq2seq systems, translation was often written as

$$p(y \mid x) \propto p(x \mid y)p(y)$$

with a separate translation model and target-language model. Neural machine translation replaced this pipeline with a single end-to-end encoder-decoder model.

1.4 Algorithms / Architectures

1. Feed-forward masked and autoregressive prediction

- Choose a fixed-size context window.
- For a masked objective, remove one token and predict it from its surrounding context.
- For an autoregressive objective, keep only the left context and predict the next token.
- Convert the visible context into embeddings or a fixed feature vector.
- Pass it through one or more feed-forward layers.
- Predict the target token with a softmax over the vocabulary.
- Repeat with a sliding window over the corpus.

2. RNN forward pass

- Initialize h_0, often to zeros.
- For each token x_t, compute h_t from the current input and previous hidden state.
- Produce $\hat{y}_t$ from h_t.
- Interpret h_t as the contextualized representation of the sequence prefix up to t.

3. RNN training

- Feed the gold token sequence one token at a time.
- Compute a prediction at each timestep.
- Evaluate cross-entropy against the gold next token.

- Sum losses across timesteps.
- Run BPTT to update W, V, U, and optionally the input embedding matrix.
- Use teacher forcing during training: feed the true previous token, not the model's sampled output.

4. Truncated BPTT

- Keep the normal recurrent forward pass.
- Backpropagate only through the most recent T timesteps.
- Update parameters periodically, for example every T tokens.
- This limits compute without reducing the forward-pass memory to an n-gram window.

5. RNN generation

- Begin with a special start token such as `<START>` or the beginning-of-sequence token `<BOS>`.
- Produce a distribution over the next token from $\hat{y}_1$.
- Sample or choose the next token.
- Feed that generated token back in as the next input.
- Continue until generating an end-of-sequence token such as `<EOS>`.

6. Seq2Seq translation

- Run the encoder over the source sentence.
- Use the final encoder state to initialize the decoder.
- Start the decoder with a start symbol.
- During training, input the gold previous target token to the decoder.
- During testing, input the decoder's own previous prediction.
- Stop when the decoder emits a stop token.

7. Attention-augmented decoding

- For each decoder step, score the current decoder state against every encoder state.
- Softmax the scores to produce attention weights.
- Compute a weighted sum of encoder states to obtain c_t.
- Combine c_t with the decoder hidden state to predict the next target token.
- Repeat for each output position, producing a different context vector at each decoder step.

1.5 Important Intuitions from the Professor

- **RNNs are the first truly contextualized embedding model in the course.** The hidden states are not just intermediates; they are the contextualized token representations.
- **Using bytes directly is possible but often an unnecessary uphill battle.** The professor emphasized that byte-level modeling makes meaning harder to recover because single bytes are not natural semantic units.
- **Teacher forcing is not a minor implementation detail.** During training, feeding model predictions back into the decoder too early would mean learning from garbage outputs instead of the correct target sequence.
- **Truncated BPTT does not mean truncated forward memory.** The model still carries recurrent state forward through long sequences; only the gradient computation is cut off.
- **Long short-term memory (LSTM) networks were a pragmatic architectural re-**

pair. They preserve the high-level RNN idea but add gating mechanisms to better preserve long-range information and mitigate vanishing gradients.

- **Basic seq2seq has a severe bottleneck.** Expecting one final encoder vector to store the entire meaning of a sentence is "wild," even if it works surprisingly well with enough data.
- **Attention is named well because it formalizes selective focus.** Instead of freezing all meaning into one vector, the decoder can revisit the source and decide what matters at each output step.
- **Attention is useful but not a perfect explanation of model behavior.** Attention heatmaps are informative, but they should not be mistaken for full causal interpretability.
- **The exact attention scorer is a design choice.** The professor noted that one can use an MLP-style scorer, a dot product, or a linear transformation, but speed matters because the score is computed many times; in practice, simple dot-product forms often win.

1.6 Examples

- **Long-range agreement example:** pronoun prediction in

  "Emily earned the top grade ... Everyone was proud of her"

  versus

  "Miguel earned the top grade ... Everyone was proud of him"

  requires preserving information from many tokens earlier.
- **Feed-forward limitation:** a fixed-window network predicting the next word in "Anqi was late for _____" can use nonlinearities but still lacks persistent memory.
- **RNN generation:** beginning with a start token, an RNN may generate text such as "Sorry Harry shouted, panicking ... " one token at a time until an end token is produced.
- **Seq2Seq translation example:**

  $$\text{The brown dog ran} \rightarrow \text{Le chien brun a couru}$$

  where the output length differs from the input length.
- **Attention weighting example:** for the first decoder word, the model may assign weights such as $0.51, 0.28, 0.14, 0.07$ over the encoder states, then form a weighted context vector.
- **Applications beyond translation:**
  - many-to-one sentiment classification,
  - one-to-one sequence tagging such as part-of-speech tagging,
  - constituency parsing,
  - image captioning with visual attention.

1.7 Common Pitfalls / Limitations

- **Feed-forward networks do not model sequential state.** Every prediction is independent except for the manually provided window.
- **RNNs are hard to parallelize.** Computation is sequential because each hidden state depends on the previous one.
- **BPTT is expensive.** Long sequences make training slow.
- **Vanishing gradients** make far-away context effectively disappear during learning.

- **Exploding gradients** destabilize optimization; gradient clipping helps numerical stability but is not a complete solution.
- **RNNs still struggle to use long-range context well in practice,** even though they are theoretically capable of representing it.
- **Vanilla seq2seq compresses the full source sequence into one vector,** which is a severe information bottleneck.
- **Attention context vectors can still ignore positional structure if built only as weighted sums.** This foreshadows the need for explicit positional mechanisms in later architectures.
- **Attention visualizations are not guaranteed explanations.** High attention to a word does not necessarily imply causal responsibility for a prediction.

1.8 Connections to Other Lectures

- This lecture directly continues Lecture 2. Word2vec produced type-level embeddings with no contextualization; RNNs turn sequence modeling into a way to learn **contextualized** token representations.
- Lecture 2's log-linear and bag-of-words models ignored order or used only shallow local windows; RNNs add persistent hidden state and shared parameters across positions.
- Seq2seq extends the one-to-one and many-to-one sequence models from this lecture into **variable-length** input/output prediction, especially for translation.
- Attention is the bridge to the next lecture: once the decoder learns to compare states and reweight all source positions, the natural next step is to remove recurrence and build models around attention itself, i.e. Transformers.
- The historical machine-translation discussion also connects back to Lecture 2's Bayes-style factorization $p(y \mid x) \propto p(x \mid y)p(y)$; seq2seq replaces the old modular pipeline with one learned end-to-end model.

1.9 Exam-Relevant Summary

- **Word token vs. word type:** RNN hidden states are contextualized token representations; word2vec embeddings are type-based.
- **Feed-forward LM weakness:** fixed window, no memory, poor parameter efficiency as context grows.
- **Feed-forward masked vs. autoregressive modeling:** masked prediction can learn contextual representations from both sides of a token, while autoregressive prediction models the next token from the left context and directly supports sequence probabilities.
- **Basic RNN equations:**

$$h_t = f(Wx_t + Vh_{t-1} + b_h), \qquad \hat{y}_t = \mathrm{softmax}(Uh_t + b_y)$$

- **Initial state:**

$$h_0 = \mathbf{0}$$

- **Training loss:**

$$\mathcal{L} = \sum_{t=1}^{T} \mathrm{CE}(y_t, \hat{y}_t)$$

- **Teacher forcing:** use gold previous tokens during training.
- **BPTT:** gradients propagate through the unrolled sequence; this is expensive and causes vanishing/exploding gradient problems.
- **Truncated BPTT:** truncate the backward pass, not the recurrent forward memory.
- **Gradient clipping:** controls exploding gradients by limiting gradient norm.
- **RNN strengths:** shared weights across positions, stateful memory, variable-length sequential processing.
- **RNN weaknesses:** slow training, poor parallelism, vanishing/exploding gradients, limited practical use of very long context.
- **Seq2Seq:** encoder reads the source sequence, decoder generates the target sequence.
- **Basic seq2seq bottleneck:** all source meaning must pass through the final encoder hidden state.
- **Popular attention scoring functions:** additive/concat, bilinear/general, and dot product.
- **Attention equations:**

$$e_{t,i} = \text{score}(h_t^{dec}, h_i^{enc}), \qquad a_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}, \qquad c_t = \sum_i a_{t,i} h_i^{enc}$$

- **Attention takeaway:** the decoder conditionally focuses on different encoder states at each output step, greatly improving seq2seq performance.
- **Historical significance:** RNN encoder-decoder models with attention were the major step before attention-only Transformer models.